

Proyecto de Software

API REST: diseñar el contrato

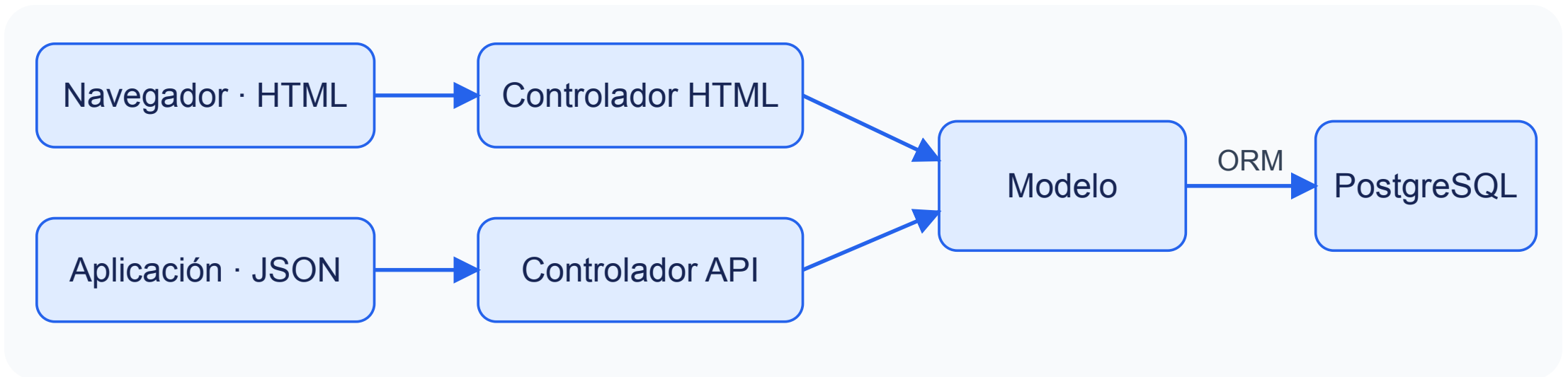
Recursos, HTTP y JSON entre aplicaciones.

Una solicitud persistida. Distintos clientes. Un contrato compartido.

Qué vamos a poder explicar

- Distinguir **recurso**, **representación** y **endpoint**.
- Elegir métodos y respuestas HTTP coherentes.
- Separar contrato JSON, reglas y persistencia.
- Diagnosticar una API, no sólo probar el caso exitoso.

El mismo modelo, otra representación



El cliente cambia. Las reglas del modelo no se duplican.

Una API es un contrato

API: interfaz que un programa ofrece a otro.

El cliente necesita conocer	El servidor debe sostener
Operación y dirección	Comportamiento definido
Datos aceptados	Validación y reglas
Respuestas posibles	Formato y significado estables

No toda API es web. No toda API web es REST.

Recurso, URL y endpoint

- **Recurso:** una solicitud, un servicio, una colección.
- **URL:** identifica el recurso expuesto.
- **Endpoint:** aquí, combinación de método y ruta.

```
GET /api/v1/solicitudes/42  
POST /api/v1/solicitudes
```

Un recurso **no tiene que corresponder a una tabla.**

JSON es una representación

```
{  
  "id": 42,  
  "servicio_id": 3,  
  "fecha": "2026-10-01",  
  "cantidad": 12  
}
```

Una fecha viaja como texto. `cantidad` viaja como número.

El objeto ORM no cruza la red. Tampoco deben hacerlo sus atributos internos.

REST no significa «CRUD con JSON»

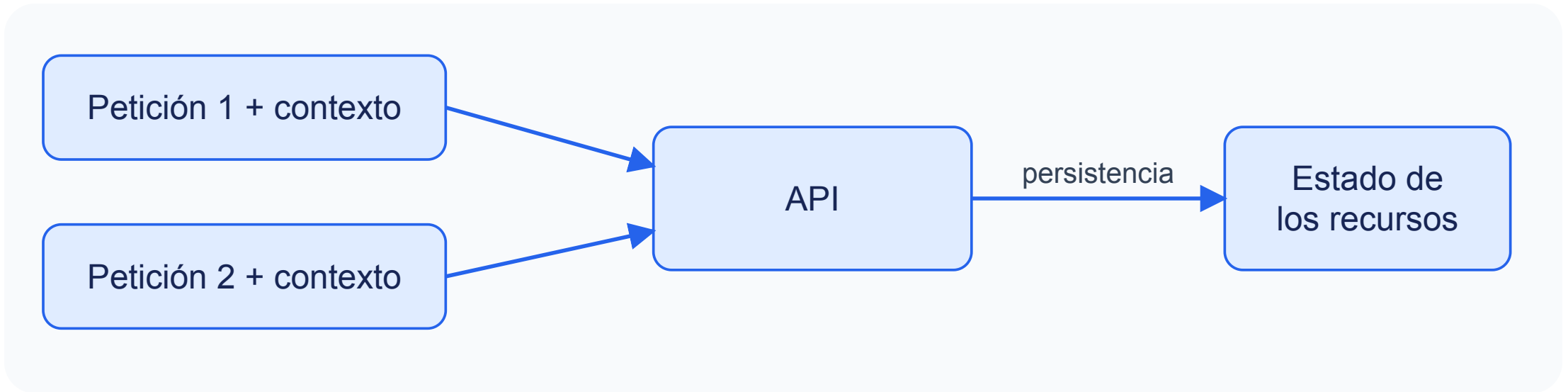
REST es un **estilo arquitectónico**:

- Cliente-servidor y solicitudes sin estado de sesión en el servidor.
- Caché, capas e interfaz uniforme.
- Código bajo demanda, opcional.

La interfaz uniforme incluye recursos, representaciones, mensajes autodescriptivos e **hipermedia**.

Usaremos una API HTTP JSON orientada a recursos; eso solo no demuestra REST completo.

Sin estado no significa sin base de datos



Estado del recurso: persiste.

Contexto de la interacción: cada petición debe aportar lo necesario.

Leer una petición HTTP

```
POST /api/v1/solicitudes HTTP/1.1
Host: localhost:5000
Content-Type: application/json
Accept: application/json

{"servicio_id": 3, "fecha": "2026-10-01", "cantidad": 12}
```

Método y destino · cabeceras · cuerpo

Es una representación textual de HTTP/1.1; no todos los mensajes llevan cuerpo.

Content-Type y Accept no son lo mismo

Cabecera	Qué comunica
Content-Type: application/json	El formato del cuerpo enviado
Accept: application/json	Qué representación acepta el cliente

En la respuesta, `Content-Type` describe **su** cuerpo.

Cambiar la cabecera no convierte un formulario en JSON.

Métodos: intención, no nombres de funciones

Método	Intención sobre el recurso
GET	Obtener una representación
POST	Procesar datos; aquí, crear en una colección
PUT	Crear o reemplazar el estado representado en esa URL
PATCH	Aplicar modificaciones parciales definidas
DELETE	Quitar la asociación del recurso con esa URL

PUT no es «PATCH con otro nombre».

Seguridad e idempotencia

Método	Seguro ¹	Idempotente por semántica
GET	Sí	Sí
PUT / DELETE	No	Sí
POST / PATCH	No	No garantizado

¹ Seguro: no solicita cambiar el estado del recurso.

Idempotente: repetir tiene el mismo **efecto pretendido** que una vez.

No exige la misma respuesta.

Checkpoint: repetir un borrado

```
DELETE /api/v1/solicitudes/42 → 204  
DELETE /api/v1/solicitudes/42 → 404
```

¿Se perdió la idempotencia?

- A.** Sí: cambiaron los códigos.
- B.** No: el efecto pretendido sigue siendo la ausencia del recurso.
- C.** DELETE nunca es idempotente.

Respuestas exitosas

Código	Uso en nuestro contrato
200 OK	Lectura o actualización con representación
201 Created	Creación; informamos la URL con Location
204 No Content	Operación exitosa sin cuerpo

```
HTTP/1.1 201 Created
Location: /api/v1/solicitudes/42
Content-Type: application/json
```

204 no lleva un JSON de confirmación.

Errores que conviene distinguir

Código	Situación
400	JSON mal formado
415	Tipo de contenido no admitido
422	JSON válido, datos que no cumplen el contrato
404	Recurso individual no encontrado
409	Conflicto con el estado actual
500	Fallo inesperado del servidor

Aquí elegimos **422 para validación** y lo documentamos.

El mapa de nuestra API

Operación	Endpoint propuesto
Listar	GET /api/v1/solicitudes
Consultar una	GET /api/v1/solicitudes/42
Crear	POST /api/v1/solicitudes
Cambiar cantidad	PATCH /api/v1/solicitudes/42
Borrar, si el dominio lo permite	DELETE /api/v1/solicitudes/42

Implementaremos **lectura individual y creación**.

Los demás endpoints definen el diseño a completar.

Colección vacía no es recurso inexistente

```
GET /api/v1/solicitudes?servicio_id=3&page=1
```

```
{  
  "items": [],  
  "page": 1,  
  "per_page": 20,  
  "next": null  
}
```

200: la colección existe, aunque no haya coincidencias.

404: no encontramos la solicitud individual pedida.

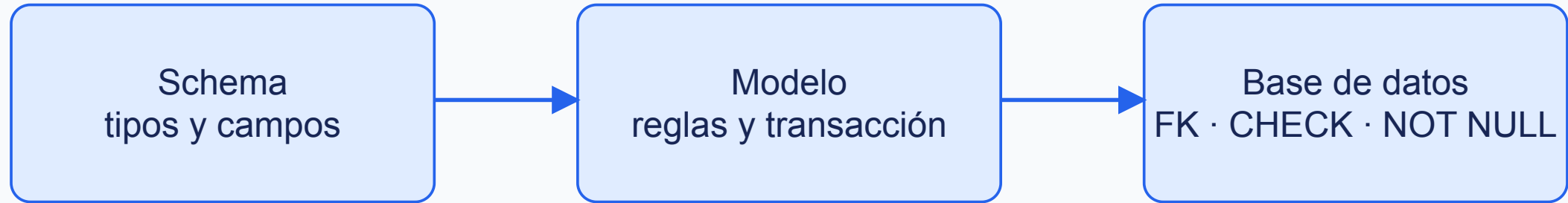
Checkpoint: diseñar una creación

Queremos crear una solicitud con servicio, fecha y cantidad.

- A. `GET /api/v1/crearSolicitud?cantidad=12`
- B. `POST /api/v1/solicitudes` + cuerpo JSON
- C. `POST /api/v1/solicitudes/42` para elegir cualquier ID

¿Qué decisión protege mejor el contrato propuesto?

Tres responsabilidades, tres garantías



El acceso autorizado se comprueba en el servidor, no en el JSON.

Un JSON válido **no prueba** que la operación esté permitida.

Marshmallow: contrato, no persistencia

```
poetry add 'marshmallow>=4,<5'
```

Sobre la aplicación Flask + SQLAlchemy de la clase anterior.

- `load` : valida y deserializa datos de entrada.
- `dump` : selecciona y serializa datos de salida.
- `jsonify` : construye la respuesta HTTP JSON.

Usaremos **Schema explícitos**, sin generar la API desde las tablas.

Entrada: sólo campos permitidos

```
# src/web/schemas/solicitudes.py
from marshmallow import Schema, fields, validate, RAISE

class SolicitudCrearSchema(Schema):
    class Meta:
        unknown = RAISE

    servicio_id = fields.Integer(
        required=True, strict=True, validate=validate.Range(min=1))
    fecha = fields.Date(required=True)
    cantidad = fields.Integer(
        required=True, strict=True, validate=validate.Range(min=1))
```

Schema de creación, no una copia de la tabla.

No aceptamos `id` ni campos extra. `"12"` no es el entero `12`.

Salida: una representación controlada

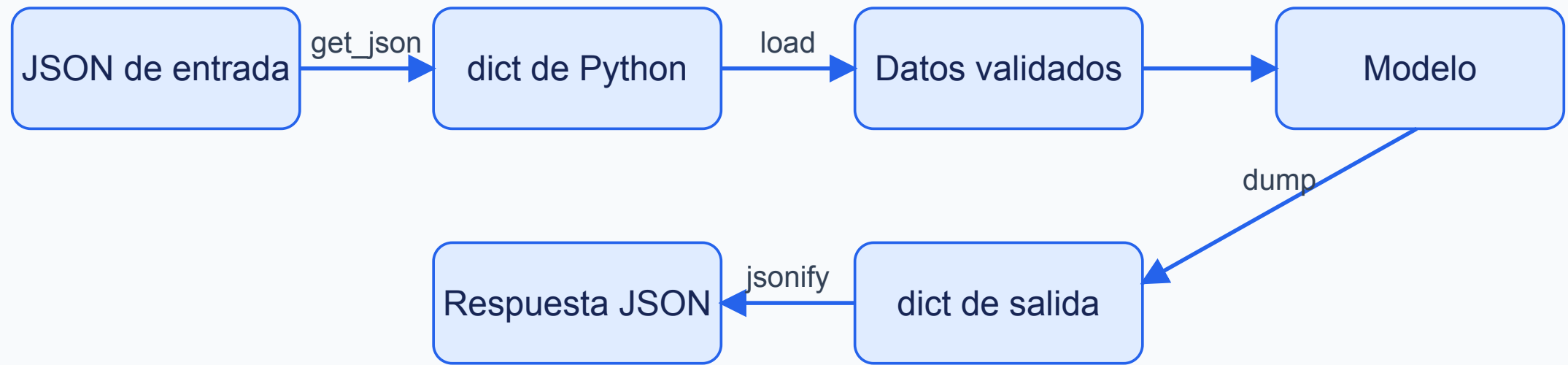
```
# Continúa src/web/schemas/solicitudes.py
class SolicitudSalidaSchema(Schema):
    id = fields.Integer()
    servicio_id = fields.Integer()
    fecha = fields.Date()
    cantidad = fields.Integer()

solicitud_crear_schema = SolicitudCrearSchema()
solicitud_salida_schema = SolicitudSalidaSchema()
```

Clases: definen campos y reglas. **Instancias:** usamos `load` / `dump` .

`dump` no reemplaza la validación del modelo.

Del JSON al modelo y de vuelta



`load` no hace `commit`. `dump` no construye una respuesta HTTP.

Un Blueprint para el contrato HTTP

```
# src/web/controllers/api_solicitudes.py
from flask import Blueprint, jsonify, request, url_for
from src.core import solicitudes
from src.web.schemas.solicitudes import (
    solicitud_crear_schema, solicitud_salida_schema,
)

api = Blueprint("api_solicitudes", __name__,
               url_prefix="/api/v1/solicitudes")
```

En la factory existente, después de configurar `db`:

```
from src.web.controllers.api_solicitudes import api
app.register_blueprint(api)
```

Los siguientes fragmentos completan estos módulos.

GET: representación o ausencia

```
# Continúa api_solicitudes.py
from werkzeug.exceptions import NotFound

@api.get("/<int:solicitud_id>")
def detalle(solicitud_id):
    obj = solicitudes.obtener(solicitud_id)
    if obj is None:
        raise NotFound("Solicitud no encontrada")
    return jsonify(solicitud_salida_schema.dump(obj)), 200
```

obtener usa db.session.get(Solicitud, solicitud_id).

El controlador traduce la ausencia al contrato HTTP.

POST: validar, crear, responder

```
# Continúa api_solicitudes.py
@api.post("")
def crear():
    datos = solicitud_crear_schema.load(request.get_json())
    obj = solicitudes.crear(**datos)
    ubicacion = url_for("api_solicitudes.detalle",
                        solicitud_id=obj.id)
    return jsonify(solicitud_salida_schema.dump(obj)), 201, {
        "Location": ubicacion
    }
```

La operación del modelo confirma antes de anunciar éxito.

Los errores se traducen fuera del camino exitoso.

El modelo conserva la transacción

```
# src/core/solicitudes.py; usa db, Servicio y Solicitud de ORM
class ServicioInexistente(Exception):
    pass

def crear(servicio_id, fecha, cantidad):
    try:
        if db.session.get(Servicio, servicio_id) is None:
            raise ServicioInexistente()
        obj = Solicitud(servicio_id=servicio_id,
                        fecha=fecha, cantidad=cantidad)
        db.session.add(obj)
        db.session.commit()
        return obj
    except Exception:
        db.session.rollback()
        raise
```

La clave foránea y el CHECK siguen siendo necesarios.

Traducir errores esperados

```
# Continúa api_solicitudes.py
from marshmallow import ValidationError

@api.errorhandler(ValidationError)
def datos_invalidos(error):
    return jsonify(error={"code": "validation_error",
                          "fields": error.messages}), 422

@api.errorhandler(solicitudes.ServicioInexistente)
def servicio_inexistente(error):
    return jsonify(error={"code": "service_not_found",
                          "fields": {"servicio_id": [
                              "El servicio no existe"]}}), 422
```

Códigos estables para el cliente; mensajes para las personas.

También los errores HTTP deben ser coherentes

```
# src/web/api_errors.py; registrar en la factory
from flask import jsonify, request
from werkzeug.exceptions import HTTPException

def registrarErroresApi(app):
    @app.errorhandler(HTTPException)
    def error_http(error):
        if not request.path.startswith("/api/"):
            return error
        respuesta = error.get_response()
        respuesta.set_data(jsonify(error={
            "code": f"http_{error.code}",
            "message": error.name}).get_data())
        respuesta.content_type = "application/json"
        return respuesta
```

Preservamos estado y cabeceras, incluido `Allow` en un 405.

Crear y seguir Location

```
curl -i http://localhost:5000/api/v1/solicitudes \
-H 'Content-Type: application/json' \
-H 'Accept: application/json' \
--data '{"servicio_id":3,"fecha":"2026-10-01","cantidad":12}'
```

Esperamos **201**, JSON y `Location` con la URL creada.

```
curl -i http://localhost:5000/api/v1/solicitudes/42
```

Usar un servicio existente y el ID realmente devuelto.

Una entrada inválida no debe crear filas

```
{"servicio_id": 3, "fecha": "2026-10-01", "cantidad": 0}
```

Respuesta esperada: **422**

```
{
  "error": {
    "code": "validation_error",
    "fields": {"cantidad": ["Must be greater than or equal to 1."]}
  }
}
```

Verificar también que la cantidad de filas **no aumentó**.

PATCH necesita su propio contrato

Diseño propuesto: sólo se modifica `cantidad` .

```
PATCH /api/v1/solicitudes/42
Content-Type: application/json

{"cantidad": 15}
```

- Ausente: no se modifica. `null` : se rechaza.
- Campos ajenos y objeto vacío: se rechazan.
- Validar, aplicar reglas y confirmar como una unidad.

Un formato JSON propio debe documentarse; no es JSON Patch.

DELETE no decide la política de conservación

```
DELETE /api/v1/solicitudes/42
```

Si está permitido y se completa: **204, sin cuerpo.**

- ¿Debe conservarse la historia?
- ¿Existen operaciones dependientes?
- ¿La intención real es cancelar, no borrar?

Una **cancelación** puede ser otro recurso o una transición explícita.

Listar sin traer toda la base

```
GET /api/v1/solicitudes?servicio_id=3&page=1&per_page=20
```

Contrato propuesto:

- Filtro validado; orden estable por `id` .
- `page ≥ 1 ; 1 ≤ per_page ≤ 100` .
- Respuesta con `items` , `page` , `per_page` y `next` .
- Página sin resultados: lista vacía.

Paginación y límites deben aplicarse **en la consulta**.

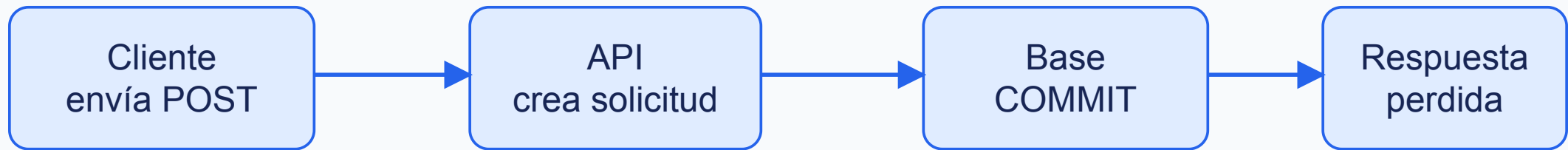
Probar el contrato con el cliente de Flask

```
# tests/test_api.py; app con base aislada y servicio_id existente
r = client.post("/api/v1/solicitudes", json={
    "servicio_id": servicio_id,
    "fecha": "2026-10-01", "cantidad": 12
})
assert r.status_code == 201
assert r.is_json
assert r.json["cantidad"] == 12

consulta = client.get(r.headers["Location"])
assert consulta.status_code == 200
assert consulta.json["id"] == r.json["id"]
```

Agregar casos de **400, 404, 405, 415 y 422**, y comprobar efectos.

Si la respuesta se pierde, ¿reintentamos?



Repetir POST sin deduplicación puede crear otra solicitud.

Un timeout no prueba que el servidor haya revertido.

Los reintentos de creación necesitan una estrategia de deduplicación.

JSON válido no significa acceso autorizado

- **Autenticación:** ¿quién realiza la petición?
- **Autorización:** ¿puede operar sobre este recurso?

Cambiar `/solicitudes/42` por `/solicitudes/43`

no debe permitir leer datos ajenos.

HTTPS, permisos por operación y recurso, límites y logs sin secretos.

Los fragmentos no implementan acceso: **no publicar así la API.**

CORS es una política del navegador

Frontend y API en **orígenes distintos**:

- El navegador puede enviar un preflight `OPTIONS`.
- El servidor declara los orígenes, métodos y cabeceras admitidos.
- Un error CORS no se corrige desactivando la seguridad del navegador.

CORS no autentica ni autoriza usuarios.

Que `curl` funcione no prueba que el navegador pueda leer la respuesta.

Documentar y evolucionar el contrato

Para cada endpoint:

Entrada · respuesta · errores · permisos · ejemplos

OpenAPI permite describir una API HTTP de forma estructurada.

- Documentar antes de que el frontend tenga que adivinar.
- Cambiar un tipo o quitar un campo puede romper clientes.
- `/v1` es una estrategia; no una obligación de REST.

Esquemas, documentación y pruebas deben mantenerse alineados.

Revisión colectiva: ¿publicaríamos esto?

```
# Propuesta deliberadamente incorrecta
@api.get("/crear")
def crear_rapido():
    obj = Solicitud(**request.get_json())
    db.session.add(obj)
    db.session.commit()
    return jsonify(obj.__dict__), 200
```

¿Qué problemas hay en **HTTP**, **entrada**, **persistencia** y **salida**?

Revisar por garantías, no por apariencia

Problema	Decisión que lo corrige
GET con efectos de creación	POST sobre la colección
Campos arbitrarios	Schema de entrada explícito
Transacción mezclada	Operación del modelo con rollback
Exposición del objeto interno	Schema de salida
Éxito ambiguo	201 y Location

Y siempre: **permisos y pruebas de errores.**

Una operación, de extremo a extremo

1. El cliente solicita una operación HTTP sobre un recurso.
2. El servidor interpreta el mensaje y controla el acceso.
3. El schema valida y deserializa la entrada.
4. El modelo aplica reglas y coordina persistencia.
5. La API devuelve una representación o un error definido.

El contrato permite colaborar sin compartir la implementación.

Lo que sigue: consumidores y acceso

Frontend: consumir representaciones y mostrar errores.

Backend: sostener contratos, permisos e integridad.

OAuth: delegar acceso sin compartir la contraseña con cada aplicación.

¿Cómo decide nuestra API quién puede hacer cada operación?

Referencias para seguir trabajando

- [Fielding: restricciones de REST](#)
- [HTTP: RFC 9110](#) · [PATCH: RFC 5789](#)
- [Flask: API](#) · [errores](#) · [pruebas](#)
- [Marshmallow: guía](#) · [campos](#)
- [Problem Details: RFC 9457](#)
- [Fetch: CORS](#) · [OpenAPI](#)